Agenda

- React Router
- useEffect
- Context

1. React Router

- It is used for component navigation
- used to add routing feature in react application
- routing feature will enable the react application to switch between the components dynamically
- provides router which does the routing
- to install the react router use the below command

```
# installation
> npm install react-router-dom
> yarn add react-router-dom
```

- To use it we need to put our main component inside
- wrap the App component inside the BrowserRouter

```
// main.jsx

import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
import { BrowserRouter } from 'react-router-dom'

createRoot(document.getElementById('root')).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
)
```

- define all the routes
- route
    - mapping between
        - path
        - element (component)

```
// App.jsx

import React from 'react'
import Login from './pages/Login/Login'
import Register from './pages/Register/Register'
```

SUNBEAM INFOTECH

```
import Home from './pages/Home/Home'
import Cart from './pages/Cart/Cart'
import Orders from './pages/Orders/Orders'
import { Route, Routes } from 'react-router-dom'

function App() {
  return (
    <div>
      <Routes>
        <Route
          path='/login'
          element={<Login />}
        />
        <Route
          path='/register'
          element={<Register />}
        />
        <Route
          path='/cart'
          element={<Cart />}
        />
        <Route
          path='/home'
          element={<Home />}
        />
        <Route
          path='/orders'
          element={<Orders />}
        />
      </Routes>
    </div>
  )
}

export default App
```

SUNBEAM INFOTECH

- BrowserRouter
  - used to add routing feature for website (browser)
  - must be the parent of in order to add the routing feature
- Routes
  - collection of Route objects
  - contains all the possible routes of an application
- Route
  - mapping between path (to launch the component) and element (component to be launched)
- Their are two ways to navigate to another component

1. static navigation

- the navigation performed without executing any JS code (performed using JSX)
- no condition is required to launch the new component
- can be performed by using `<Link></Link>`

```
function Login() {
  return (
    <div>
      <h2>Login</h2>
      <Link to='/register'>Register here</Link>
    </div>
  )
}
```

2. dynamic navigation

- navigation is performed executing the JS code
- also known as conditional navigation
- can be achieved by a react hook named useNavigate()

```
import { useNavigate } from 'react-router-dom'

function Register() {
  // get the reference of navigate function
  const navigate = useNavigate()

  const onRegister = () => {
    // navigate to login page
    navigate('/login')
  }

  return (
    <div>
      <h2>Register</h2>
      <button onClick={onRegister}>Login</button>
    </div>
  )
}

export default Register
```

SUNBEAM INFOTECH

- nested routing
  - routing with parent-child relationship
  - used to share contents with all child components
  - `<Outlet />` is a placeholder to load the child component based on the route or path
  - the child component can be loaded using the path format parent-path/child-path

```
// Home.jsx
import React from 'react'
import { Outlet } from 'react-router-dom'
function Home() {
  return (
    <div>
    <h1>Home</h1>
    <Outlet />
    </div>
    )
}
export default Home
```

react toastify

- used to show the toast message

```
# installation
> npm install react-toastify
> yarn add react-toastify
```

```
import { ToastContainer } from 'react-toastify'

function App() {
  return (
    <div>
      {/* add the toast container to show the toast messages */}
      <ToastContainer />
    </div>
  )
}

export default App
```

```
import { toast } from 'react-toastify'

function Dummy() {
  const onShow = () => {
    toast.success('this is a success message')
  }

  return (
    <div>
      <button onClick={onShow}>show success</button>
    </div>
```

SUNBEAM INFOTECH

```
  )
}
```

Use Effect

- used to handle component life cycle events
- accepts 2 parameters
  - 1st: function reference which will be called in different events
  - 2nd: dependency array
- component mounted/launched event
  - this event occurs only once when the component gets loaded/mounted
  - the dependency array must be empty

```
import { useEffect } from 'react'

function App() {
  useEffect(() => {
    console.log('component got loaded')
  }, [])

  return (
    <div>
      <h2>App component</h2>
    </div>
  )
}
```

- component getting unmounted/unloaded
  - this event is fired only once when the component is unmounted/unloaded
  - dependency array must be empty

```
import { useEffect } from 'react'

function App() {
  useEffect(() => {
    console.log('component got loaded')

    return () => {
      console.log('component got unloaded')
    }
  }, [])

  return (
    <div>
      <h2>App component</h2>
    </div>
```

SUNBEAM INFOTECH

```
  )
}
```

- a state is changed is updated because of dependency

```
import { useEffect, useState } from 'react'

function App() {
  const [counter, setCounter] = useState(0)

  // when there is change in the counter, the useEffect will be called
  useEffect(() => {
    console.log('updated counter')
  }, [counter])

  return (
    <div>
      <h2>App component</h2>
    </div>
  )
}
```

- component state changed
  - required to perform an action when any of the component state is changed
  - dependency array must not be provided

```
import React, { useEffect, useState } from 'react'

function Counter() {
const [counter1, setCounter1] = useState(0)
const [counter2, setCounter2] = useState(0)

useEffect(() => {
// this function gets called when there is change in state
console.log(`state changed`)
})

  return (
    <div>
    <h1>Counter</h1>
    <div>counter1: {counter1}</div>
    <div>counter2: {counter2}</div>
    <button onClick={() => setCounter1(counter1 + 1)}>
            update counter1
    </button>
    <button onClick={() => setCounter2(counter2 + 1)}>
            update counter2
    </button>
    </div>
```

SUNBEAM INFOTECH

```
    )
}
export default Counter
```

useContext()

- used to use context in an application
- context
    - information which is required to shared with multiple components without props drilling
    - It is a solution for props drilling
- props drilling
    - sharing information from one component with multiple (other or child components) components
    - try to avoid it as it will introduce many bugs

```
// Counter.jsx
import { CounterContext } from './App'

function Counter() {
  // read the members from context
  const { counter, setCounter } = useContext(CounterContext)

  return (
    <div>
      <div>counter = {counter}</div>
      <button onClick={() => setCounter(counter + 1)}>update</button>
    </div>
  )
}
```

- NOTE
    - the components with which you want to share the information, must be declared inside the Providers' list
    - if a component is not in the Providers list, the useContext() returns undefined

```
// App.jsx

import { useState, createContext } from 'react'
import Counter from './Counter'

// create context
export const CounterContext = createContext()

function App() {
  const [counter, setCounter] = useState(0)

  return (
    <div>
      <CounterContext.Provider value={{ counter, setCounter }}>
```

SUNBEAM INFOTECH

```
          <Counter />
        </CounterContext.Provider>
      </div>
    )
  }
```

Q.Session storage vs context

- The session storage will share the data will all components but will not help in render
- Context helps in render automatically
- After refresh the data still remains same in session storage however it resets after refresh if kept in context
- As per the requirement the out of these we should be using.

SUNBEAM INFOTECH